

操作系统

第二章 进程管理

0、程序的执行



一、应用程序的 逻辑段

- 代码段：应用程序中定义的所有函数
- 数据段：带初值的全局变量
- bss段： 不带初值的全局变量
- 只读数据段： 常量
- 堆：分配给malloc的动态内存（c++是new）
- 栈：用来存放局部变量



例：matrix程序的逻辑段

```
# include <stdlib.h>
# include <stdio.h>
```

```
# define M 16
int version = 1;
int matrixOriginal[M][M];
int matrixDes [M][M];
```

```
void produce ( int row, int column )
{
    int i;
    for (i=0;i<M;i++)
        matrixDes[row][column] +=
            matrixOriginal[row][i] * matrixOriginal[i][column];
}
```

```
int main()
{
    int i , j;

    for (i=0;i<M;i++)
        for (j=0;j<M;j++) {
            matrixOriginal[i][j] = 1;
            matrixDes[i][j] = 0;
        }

    for (i=0;i<M;i++)
        for (j=0;j<M;j++)
            produce(i , j);

    printf("matrixDes == \n");

    for (i=0;i<M;i++) {
        for (j=0;j<M;j++)
            printf("%d\t", matrixDes[i][j]);
        printf("\n");
    }
}
```

代码段

main()
produce()
printf.....

堆
无

数据段

version(4字节)

只读数据段

matrixDes == \n\0
%d\t\0
\n\0

bss段

matrixOriginal, matrixDes
(共计 $16*16*4*2 = 2048$ 字节)

栈

i, j



二、CPU、内存 和 程序执行

1、指令 和 数据的地址

- 编译器 编译、链接 源程序时确定指令和数据的地址，这个过程叫编址
- 编译器对所有应用程序使用相同的编址规则。以Unix V6++系统的编译器MinGW为例：
 - 先排代码段，再排数据段，只读数据段，bss。
 - 随后是应用程序的堆（不执行malloc的程序，没这个）。
 - 最后是用户栈，在地址的最高端。
- 从所有源程序中收集函数，线性排列，随后对所有指令编址。MinGW，代码段起始地址0x401000，所以，
 - 第一条指令地址是0x401000，
 - 第二条指令地址是0x401000 + 第一条指令长度
 -

代码段长度0x4000字节向上取整
copy程序，0x3000字节

- 之后，从所有源程序中收集全局变量，线性排列，对它们编址。本例，
 - 数据段首地址 0x404000，长度 0x1000
 - 全局变量 version 的地址是 0x404000
- 只读数据段
 - 首地址0x405000，长度 0x1000
 - printf 格式化字符串的首地址在这个区间
- bss段
 - 首地址0x406000，长度 0x1000
 - 矩阵matrixOriginal的首地址0x406000， matrixDes的首地址0x406400
 - 习题：求数组元素matrixOriginal[2][1]的首地址

备注：代码段，数据段，只读数据段，bss中的代码数据，编译器确定地址。它们是程序的静态部分。

- 堆 和 栈，是动态空间。程序执行时，按需分配，用完释放。
 - 不执行malloc的程序没有堆。动态内存，堆上分配。
 - 程序执行malloc函数分配动态内存，使用完毕调用函数free，将其释放。
 - 所有程序都有自己的栈。局部变量，栈上分配。
 - 栈由栈帧组成，每个栈帧对应一个子程序，存放入口参数、局部变量，返回地址.....
 - 栈底，main栈帧；其上，main调用的子程序a；再其上，a调用的子程序b
 - 函数调用，压入栈帧；函数返回，弹出栈帧。所以，函数返回后，它的局部变量就不可访问了
 - Unix V6++，栈底8M字节

2、程序的地址空间和 内存布局 (Memory Layout)

- 应用程序，所有指令 和 数据地址，构成地址空间。
- 地址空间中，同一逻辑段的指令或数据连续存放。不同逻辑段在地址空间中的排列次序，叫做程序的内存布局。
- 编译器为一个具体的计算机系统，编译的所有应用程序，内存布局相同，区别仅在于各逻辑段的长度。
- 不同编译器，程序内存布局不相同。对概念理解没有影响。



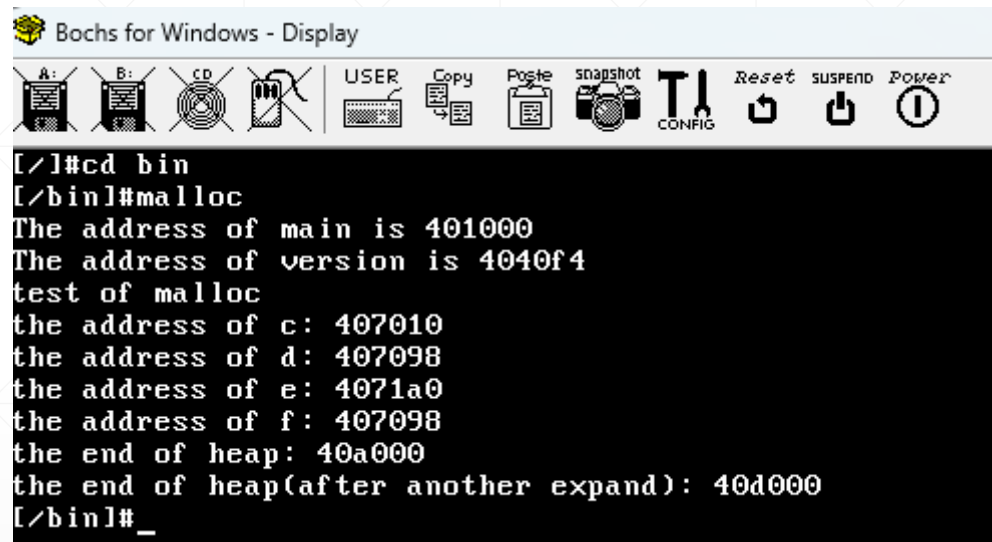
Unix V6++应用程序的内存布局



程序matrix的内存布局

例：观察Unix V6++，应用程序的内存布局

```
1 #include <stdio.h>
2 #include <stdlib.h>
3 #include <malloc.h>
4 #include <sys.h>
5
6 int version = 1;
7
8 int main1()
9 {
10     int i, j;
11
12     printf("The address of main is %x\n", &main1);
13
14     printf("The address of version is %x\n", &version);
15
16     printf("test of malloc\n");
17     int *c = malloc(128);
18     printf("the address of c: %x\n", c);
19     int *d = malloc(256);
20     printf("the address of d: %x\n", d);
21     int *e = malloc(256);
22     printf("the address of e: %x\n", e);
23     free(d);
24     int *f = malloc(83);
25     printf("the address of f: %x\n", f);
26
27     printf("the end of heap: %x\n", sbrk(0));
28
29     f = malloc(3*4096);
30     printf("the end of heap(after another expand): %x\n", sbrk(0));
31
32     return 0;
33 }
```

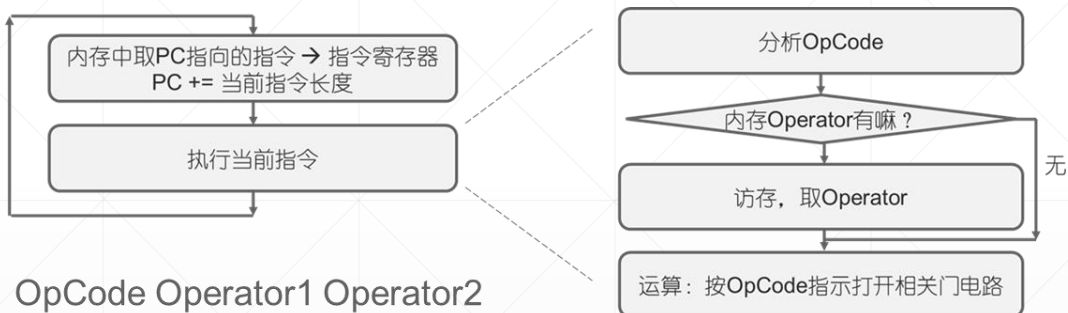


```
[/l]#cd bin
[/bin]#malloc
The address of main is 401000
The address of version is 4040f4
test of malloc
the address of c: 407010
the address of d: 407098
the address of e: 4071a0
the address of f: 407098
the end of heap: 40a000
the end of heap(after another expand): 40d000
[/bin]#_
```

提问：局部变量 i, j 的值大约是多少？

修改malloc.c程序，输出 i, j 变量的值，验证你的判断。

3、程序执行



- 取指
 - PC是下条指令地址，CPU把它放在地址总线上，发内存读命令
 - 一个主存周期后，CPU读数据总线，get 当前需要执行的指令：OpCode Operator1 Operator2
- CPU译码、执行指令（同时 PC+当前指令的长度 指向下条指令）
- 读内存变量
 - 全局变量。~的地址是常数，放地址总线，发内存读命令。例，访问version变量，CPU将地址常数 0x404000 送地址总线。一个主存周期后，CPU 得到version变量的值：1。bss，只读变量，访存方法类似。
 - 局部变量。~的地址，运行时计算得出
- 写内存变量
 - 内存变量地址→地址总线，值→数据总线，发内存写命令
 - 一个主存周期后，数据总线上的值写入内存变量

程序执行需要使用CPU寄存器 (以 Intel i386 芯片为例)

- 寄存器是CPU内部的存储单元，容量极小，访问速度极快。
 - EIP，下条指令地址 ← 控制程序执行流程
 - EBP，栈顶帧基地址
 - ESP，栈顶
 - EAX，子程序的返回值
 - EBX、ECX、EDX，其它辅助运算作用
 - ESI、EDI，其它辅助运算作用（常见于字符串操作）
 - EFLAGS，程序状态字 ← 记录程序执行状态，比如运算结果是否为0
- 供编译器维护栈空间



Intel i386 指令

- 运算

- add EAX, 5
- sub EBX, 1
- add var, EAX
- sub var, ECX

- 赋值

- mov par, 5
- mov var, EAX
- mov ESI, par

特殊指令 1：使用堆栈，push、pop指令

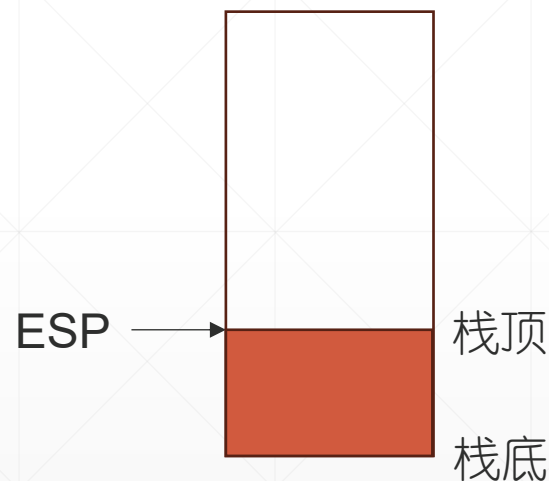
- 堆栈，后进先出。ESP，始终指向栈顶单元。

- 压栈 push EAX

- 硬件逻辑 $ESP -= 4$; $EAX \rightarrow [ESP]$

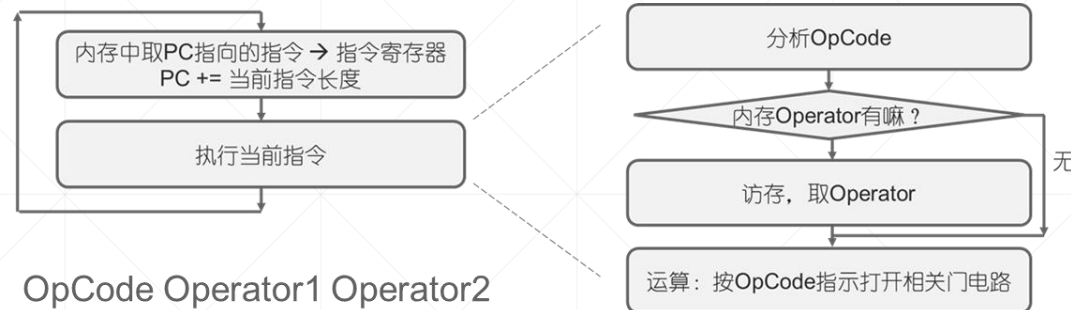
- 出栈 pop EAX

- 硬件逻辑 $[ESP] \rightarrow EAX$; $ESP += 4$;



特殊指令 2：程序的控制

- 顺序执行
- 跳转指令 `jmp label`
 - 硬件逻辑: `label → EIP`
- 子程序调用指令 `call sub`
 - 硬件逻辑: `push EIP (call指令下条指令的地址压栈) , sub → EIP`
- 子程序调用指令 `ret`
 - 硬件逻辑: `pop EIP (前提, ESP指向sub函数的返回地址)`



注：EIP就是PC

子程序调用与返回

- 高级程序设计语言为每次函数调用分配一个栈帧。栈帧存放在栈空间，是一块独立的内存单元。用来存放该函数的入口参数 和 局部变量。
- 函数返回，销毁栈帧，入口参数和局部变量从内存中消失。

CPU增设以下机制，供编译器维护栈空间

1、通用寄存器组，增设

ESP, 栈顶指针

EBP, 栈帧基址

EAX, 存放函数的返回值

2、指令集，增设

push, 压栈

pop, 弹出栈顶单元的值

ret, 函数返回

以 main() 调用 a函数为例

```
int main(int argc, char**argv)
{
    int x, y;
    x=1;
    y = a(x);
    L: .....
}
```



```
int a( int i)
{
    int b = ++i;
    return b;
}
```



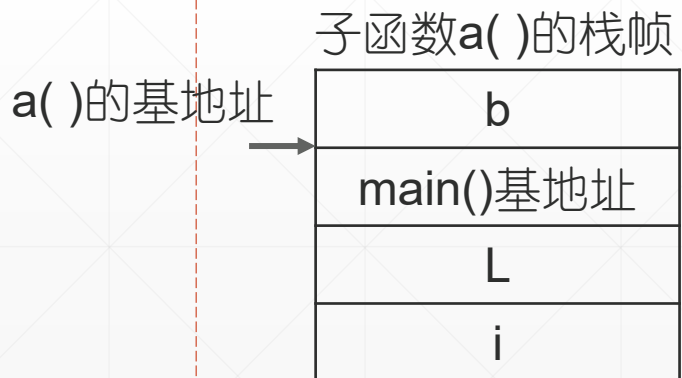
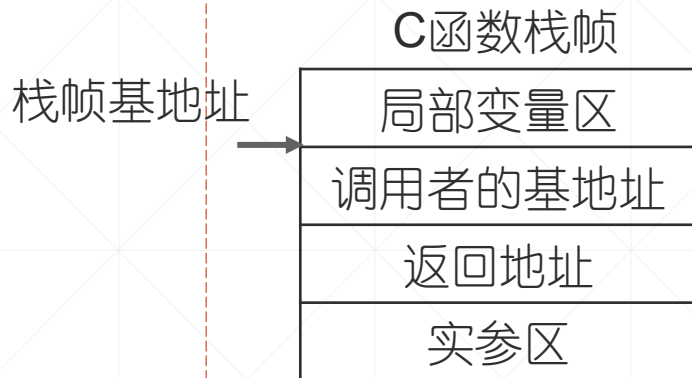
栈帧的细节

```
int main(int argc, char**argv)
```

```
{
    int x, y;
    x=1;
    y = a(x);
    L: .....
}
```

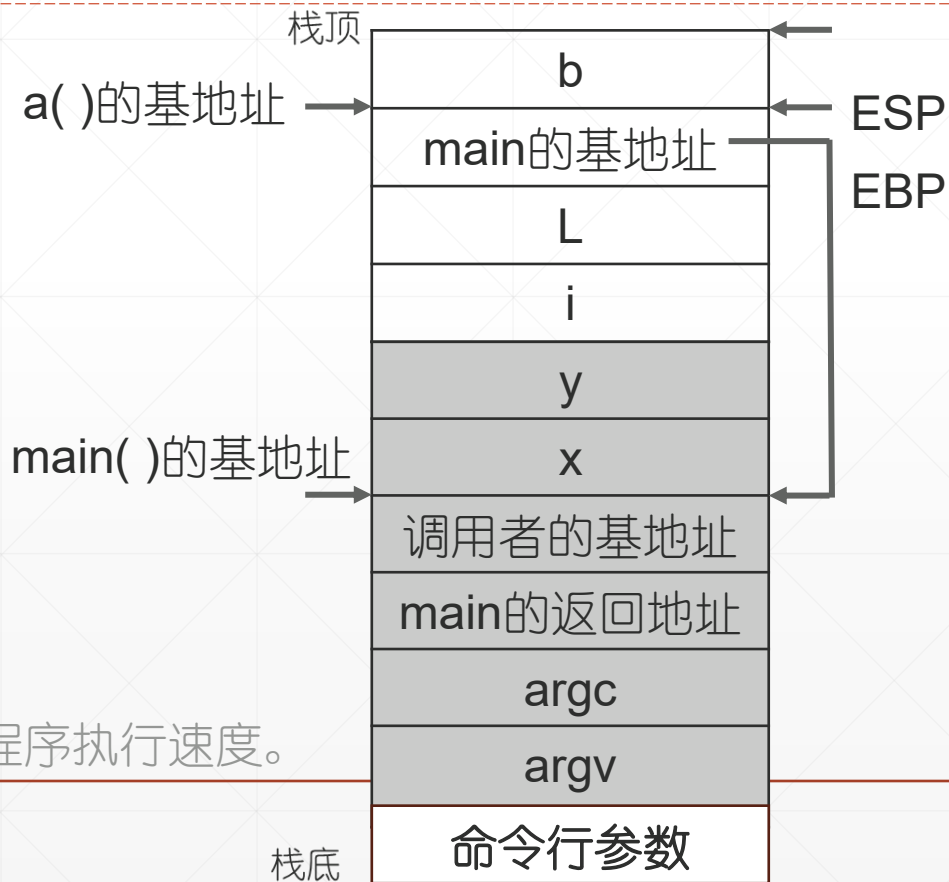
```
int a( int i)
{
    int b = ++i;
    return b;
}
```

ESP, 栈顶指针
EBP, 栈顶帧基地址
EAX, 函数返回值



局部变量的表示:
[EBP-4], 第1个局部变量
[EBP-8], 第2个局部变量
.....

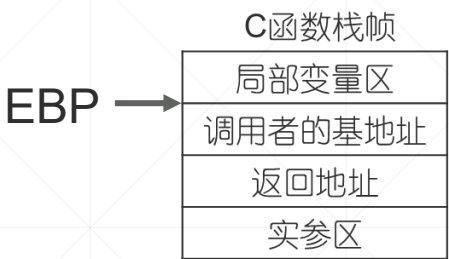
实参的表示:
[EBP+8], 第1个实参
[EBP+12], 第2个实参
.....



备注：实际执行时栈帧会对齐缓存行，提高程序执行速度。



子程序调用、返回



局部变量的表示
[EBP-4], 第1个局部变量
[EBP-8], 第2个局部变量
.....

形参的表示
[EBP+8], 第1个形参
[EBP+12], 第2个形参
.....

main函数执行时
[EBP-4], x
[EBP-8], y
[EBP+8], argc
.....

a函数执行时
[EBP-4], b
[EBP+8], 形参 i
.....

```
int main(int argc, char**argv)
{
    int x, y;
    x=1;
    y = a(x);
    .....
}
L1:  push -0x4(%ebp)
L2:  call a
L12: add 0x4, %esp
L13: mov %eax, -0x8(%ebp)
.....
```

```
int a( int i)
{
    int b = ++i;
    return b;
}
```

```
L3:  push  %ebp
L4:  mov   %esp,%ebp
L5:  sub   $0x4,%esp
L6:  incl  0x8(%ebp)
L7:  mov   0x8(%ebp),%eax
L8:  mov   %eax,-0x4(%ebp)
L9:  mov   -0x4(%ebp),%eax
L10: leave
L11: ret
```

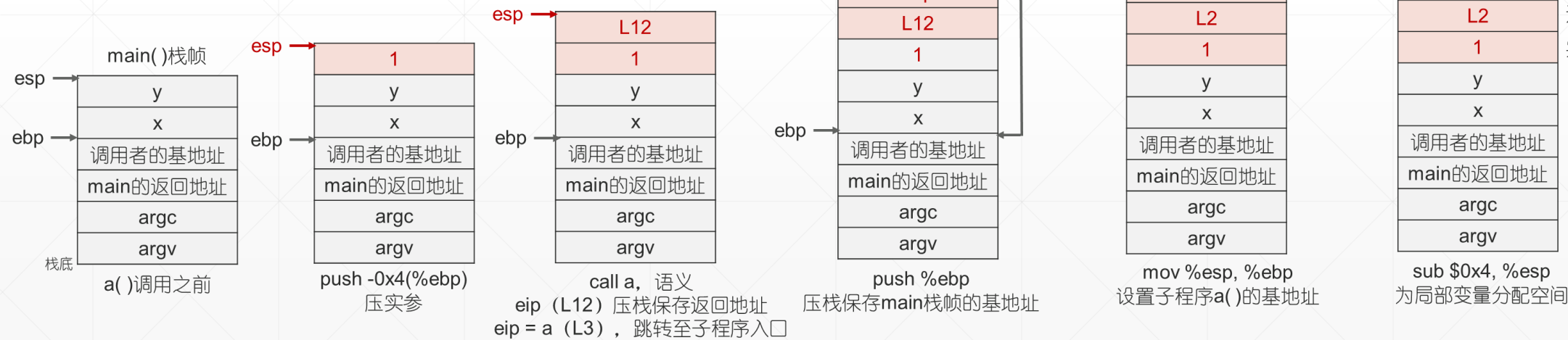
调用a()子程序，传参与栈帧构造 (L1~L5)

```
int main(int argc, char**argv)
{
    int x, y;
    x=1;
    y = a(x);
    .....
}
```

```
.....
L1:  push -0x4(%ebp)
L2:  call a
L12: add 0x4, %esp
L13: mov %eax, -0x8(%ebp)
.....
```

```
int a( int i)
{
    int b = ++i;
    return b;
}
```

```
L3:  push %ebp
L4:  mov  %esp,%ebp
L5:  sub  $0x4,%esp
L6:  incl 0x8(%ebp)
L7:  mov  0x8(%ebp),%eax
L8:  mov  %eax,-0x4(%ebp)
L9:  mov  -0x4(%ebp),%eax
L10: leave
L11: ret
```



局部变量
调用者基地址
返回地址
实参区

至此，a()栈帧构造完成，CPU可以基于ebp用间接寻址方式访问a()的实参和局部变量。

注，压栈时，先压第二个实参，后压第一个实参

子程序返回，栈帧销毁 (L10~L12)

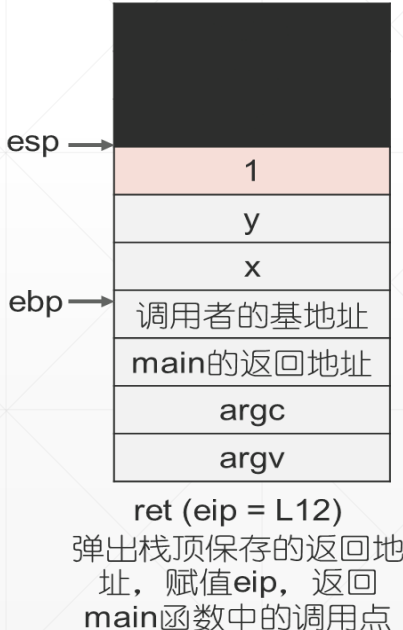
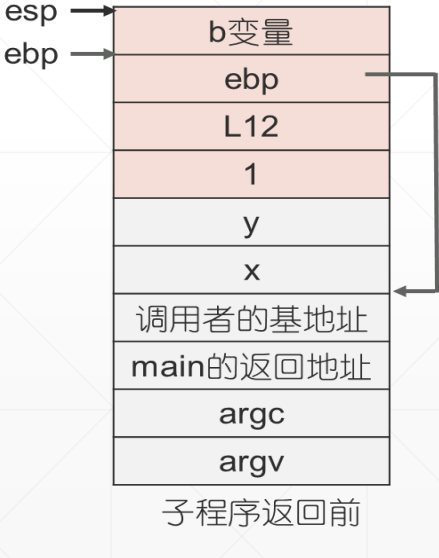
```
int main(int argc, char**argv)
{
    int x, y;
    x=1;
    y = a(x);
    .....
}
```

```
.....
L1:  push -0x4(%ebp)
L2:  call a
L12: add 0x4, %esp
L13: mov %eax, -0x8(%ebp)
.....
```

```
int a( int i)
{
    int b = ++i;
    return b;
}
```

leave等价于
mov %ebp,%esp
pop %ebp

```
L3:  push %ebp
L4:  mov  %esp,%ebp
L5:  sub  $0x4,%esp
L6:  incl 0x8(%ebp)
L7:  mov  0x8(%ebp),%eax
L8:  mov  %eax,-0x4(%ebp)
L9:  mov  -0x4(%ebp),%eax
L10: leave
L11: ret
```



至此，a()栈帧
清理完毕，恢复
调用前的状态

子程序返回，传递返回值 (L9, L13)

L9, 子函数a()返回前将返回值存入EAX寄存器。

L13, 返回后main()从EAX中获得函数的返回值，赋值局部变量y。

```
int main(int argc, char**argv)
{
    int x, y;
    x=1;
    y = a(x);
    .....
}
```

```
.....
L1:  push -0x4(%ebp)
L2:  call a
L12: add 0x4, %esp
L13: mov %eax, -0x8(%ebp)
```

.....

y

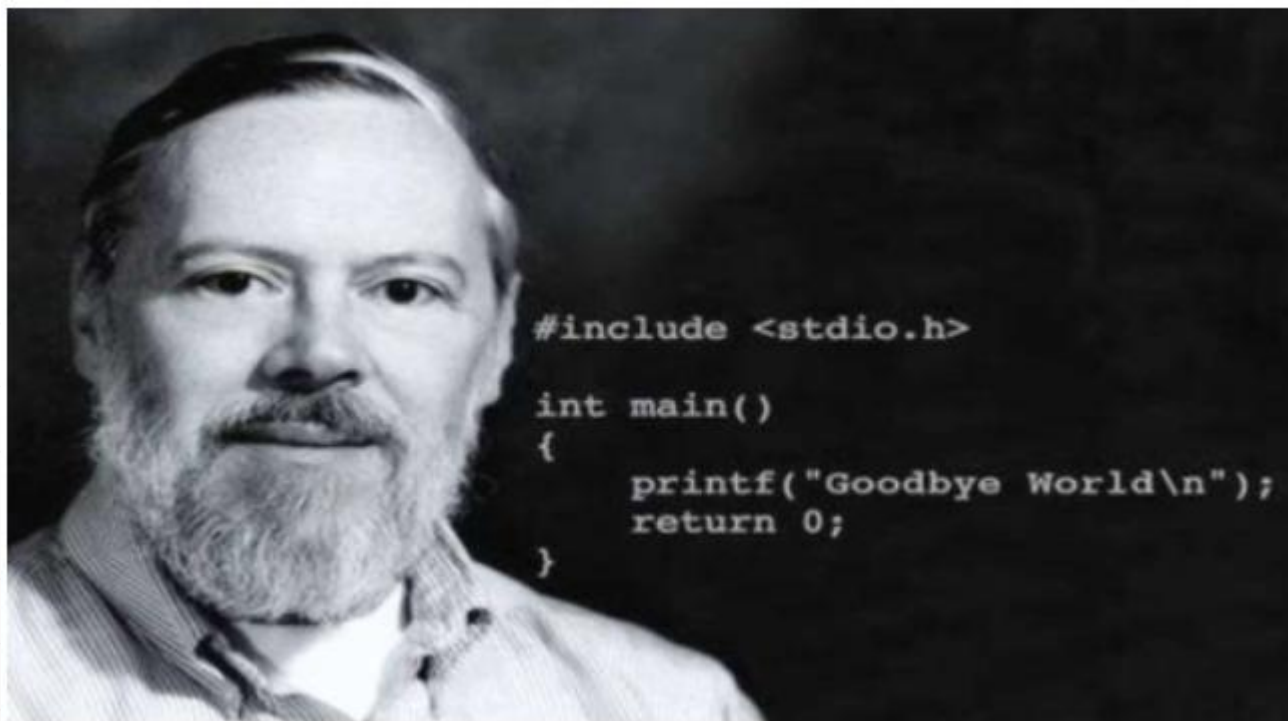
```
int a( int i)
{
    int b = ++i;
    return b;
}
```

```
L3:  push  %ebp
L4:  mov   %esp,%ebp
L5:  sub   $0x4,%esp
L6:  incl  0x8(%ebp)
L7:  mov   0x8(%ebp),%eax
L8:  mov   %eax,-0x4(%ebp)
L9:  mov   -0x4(%ebp),%eax
L10: leave
L11: ret
```

b

对应 b=++i, 观察局部变量和实参的表示和使用。

C语言是为Unix V6量身定制的高级语言



里奇，C语言之父。

4、可执行文件

- 每个可执行文件是一个程序。

- 格式:

程序头, 代码段, 数据段, 只读数据段, 符号表

- 程序头是一个数据结构, 描述程序地址空间中的逻辑段。

```
D:\UNIX V6++V1\oos\tools>objdump -h showStack.exe

showStack.exe:      file format pei-i386

Sections:
Idx Name          Size      VMA       LMA       File off  Algn
 0 .text          000020e8  00401000  00401000  00000400  2**2
   CONTENTS, ALLOC, LOAD, READONLY, CODE
 1 .data           0000000c  00404000  00404000  00002600  2**2
   CONTENTS, ALLOC, LOAD, DATA
 2 .rdata          00000204  00405000  00405000  00002800  2**3
   CONTENTS, ALLOC, LOAD, READONLY, DATA
 3 .bss            00000020  00406000  00406000  00000000  2**2
   ALLOC
```

```
#include <stdio.h>
int version=1;

main1()
{
    int a,b,result;
    a = 1;
    b = 2;
    result = sum(a,b);
    printf("result=%d\n",result);
}

int sum(var1, var2)
{
    int count;
    version = 2;
    count = var1 + var2;
    return(count);
}
```



- 需要执行应用程序时，读可执行文件的程序头。
- 为代码段分配足够内存，复制可执行文件中的代码段。
- ~数据段~~~~~，为全局变量赋初值。
- ~只读数据段~~~~~，为常量赋值。
- ~bss~~~~~，清0。 为赋值全局变量，初值为0。
- ESP = 8M。push 压入命令行参数
- call main，程序开始跑。

作业

- 实验二 Unix V6++ 程序栈帧和内存布局，观察可执行程序。